

**LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 4 ACARA 8
SEMESTER 3**



**Tugas :
PDO, Prepared Statement, Relasi,
dan CRUD Lengkap**

**Disusun Oleh:
Uaryl Engno Laurenca (E41250317)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

A. PENDAHULUAN

Sebuah aplikasi web yang berinteraksi dengan database sangat rentan terhadap serangan SQL Injection, misalnya ketika seorang pengguna mengetik ' OR '1'='1 pada kolom pencarian. Apabila query dibangun dengan menggabungkan string secara langsung, seluruh data bisa bocor karena input berbahaya tersebut ikut dieksekusi sebagai bagian dari perintah SQL. Prepared statement pada PDO mengatasi masalah ini dengan memisahkan struktur query dan data, sehingga input apa pun dari pengguna selalu diperlakukan sebagai data biasa, bukan sebagai perintah SQL.

Pada praktikum Acara 8 ini, dibangun operasi CRUD (Create, Read, Update, Delete) yang lengkap untuk tiga resource sekaligus Mahasiswa, Prodi, dan Mata Kuliah menggunakan class `Database` bertipe *singleton* untuk mengelola satu koneksi PDO, prepared statement pada seluruh query, serta `JOIN` untuk menampilkan relasi antar tabel (misalnya nama prodi pada daftar mahasiswa).

B. TUJUAN PRAKTIKUM

1. Mengimplementasikan class `Database` sebagai *singleton* yang mengelola satu koneksi PDO untuk seluruh aplikasi.
2. Membuat method pada Model (`all`, `find`, `create`, `update`, `delete`) menggunakan prepared statement.
3. Melengkapi Controller dan View untuk CRUD penuh pada resource Mahasiswa.
4. Menerapkan pola yang sama untuk resource Prodi dan Mata Kuliah.
5. Menampilkan relasi antar tabel menggunakan `JOIN`, khususnya menampilkan nama prodi pada daftar mahasiswa dan mata kuliah.
6. Menambahkan konfirmasi JavaScript (`confirm()`) sebelum proses hapus data dijalankan.
7. Menambahkan fitur pencarian (`search`) pada halaman index mahasiswa berdasarkan nama atau NIM menggunakan `LIKE` dengan prepared statement, sebagai tugas mandiri.

C. DASAR TEORI

PDO (*PHP Data Objects*) merupakan ekstensi PHP yang digunakan untuk mengakses database. Pada aplikasi ini, koneksi PDO dikelola melalui class `Database` dengan pola Singleton, sehingga hanya terdapat satu instance koneksi yang digunakan oleh seluruh aplikasi melalui method `getInstance()`.

Prepared statement menggunakan `$pdo->prepare()` dan `execute()` dengan parameter seperti `:nim` untuk memisahkan query dan data. Pengaturan `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` digunakan agar kesalahan database dapat ditangani sebagai *exception*, sedangkan `PDO::ATTR_EMULATE_PREPARES => false` membuat prepared statement dijalankan langsung oleh database sehingga lebih aman dari SQL Injection.

Transaction digunakan ketika beberapa query harus berhasil secara keseluruhan. Proses dilakukan menggunakan `beginTransaction()`, `commit()`, dan `rollBack()`. Jika terjadi kesalahan pada salah satu query, seluruh perubahan dapat dibatalkan agar

Relasi *One-to-Many* terjadi ketika satu Prodi memiliki banyak Mahasiswa. Untuk menampilkan nama prodi pada data mahasiswa, digunakan `JOIN` antara tabel mahasiswa dan prodi berdasarkan `prodi_id`, sehingga data dapat ditampilkan dalam satu query. Setiap resource memiliki fungsi CRUD berupa `index`, `create`, `store`, `edit`, `update`, dan `destroy`. Operasi yang menambah, mengubah, atau menghapus data menggunakan method `POST`, sehingga perubahan data tidak dilakukan hanya melalui akses URL dengan method `GET`.

D. HASIL DAN PEMBAHASAN

1. Implementasi Database Singleton

```
ACARAB > app > Core > Database.php
1  <?php
2
3  namespace App\Core;
4
5  use PDO;
6  use PDOException;
7
8  class Database
9  {
10     // static: satu-satunya instance PDO dipakai bersama di seluruh aplikasi
11     private static ?PDO $instance = null;
12
13     public static function getInstance(): PDO
14     {
15         if (self::$instance === null) {
16             $config = require __DIR__ . '/../config/database.php';
17             $dsn = "mysql:host={$config['host']};dbname={$config['dbname']};charset={$config['charset']}";
18
19             try {
20                 self::$instance = new PDO($dsn, $config['username'], $config['password'], [
21                     PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
22                     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
23                     // Matikan emulasi prepared statement -> query dan data benar-benar dikirim terpisah ke MySQL
24                     PDO::ATTR_EMULATE_PREPARES => false,
25                 ]);
26             } catch (PDOException $e) {
27                 die("Koneksi database gagal: " . $e->getMessage());
28             }
29         }
30
31         return self::$instance;
32     }
33 }
```

Class `Database` dibuat dengan property statis `private static ?PDO $instance = null`. Method `getInstance()` memeriksa apakah instance sudah ada; jika belum, koneksi PDO baru dibuat berdasarkan konfigurasi dari `config/database.php` dan disimpan ke property statis tersebut, sehingga pemanggilan berikutnya cukup mengembalikan koneksi yang sama tanpa membuat koneksi baru.

2. Implementasi Model dengan Prepared Statement

```
ACARA8 > app > Models > MahasiswaModel.php
1  <?php
2
3  namespace App\Models;
4
5  use App\Core\Database;
6  use PDO;
7
8  class MahasiswaModel
9  {
10     private PDO $pdo;
11
12     public function __construct()
13     {
14         $this->pdo = Database::getInstance();
15     }
16
17     // Tugas Mandiri: parameter $keyword untuk pencarian nama/NIM
18     public function all(string $keyword = ''): array
19     {
20         $sql = "SELECT m.*, p.nama AS prodi_nama
21             FROM mahasiswa m
22             JOIN prodi p ON m.prodi_id = p.id";
23
24         $params = [];
25
26         if ($keyword !== '') {
27             // Dua placeholder berbeda, karena PDO (emulate prepares = false)
28             // tidak mengizinkan satu nama parameter dipakai dua kali
29             $sql .= " WHERE m.nama LIKE :keyword_nama OR m.nim LIKE :keyword_nim";
30             $params['keyword_nama'] = '%' . $keyword . '%';
31             $params['keyword_nim'] = '%' . $keyword . '%';
32         }
33
34         $sql .= " ORDER BY m.nama ASC";
35
36         $stmt = $this->pdo->prepare($sql);
37         $stmt->execute($params);
38         return $stmt->fetchAll();
39     }
40
41     public function find(int $id): ?array
42     {
43         $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id = :id");
44         $stmt->execute(['id' => $id]);
45         $row = $stmt->fetch();
46         return $row ? : null;
47     }
48
49     public function create(array $data): void
50     {
51         $stmt = $this->pdo->prepare(
52             "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan)
53             VALUES (:nim, :nama, :email, :prodi_id, :angkatan)"
54         );
55         $stmt->execute($data);
56     }
57
58     public function update(int $id, array $data): void
59     {
60         $data['id'] = $id;
61         $stmt = $this->pdo->prepare(
62             "UPDATE mahasiswa
63             SET nim = :nim, nama = :nama, email = :email, prodi_id = :prodi_id, angkatan = :angkatan
64             WHERE id = :id"
65         );
66         $stmt->execute($data);
67     }
68
69     public function delete(int $id): void
70     {
71         $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
72         $stmt->execute(['id' => $id]);
73     }
74 }
```

```

ACARAB > app > Models > ProdiModel.php
1  <?php
2
3  namespace App\Models;
4
5  use App\Core\Database;
6  use PDO;
7
8  class ProdiModel
9  {
10     private PDO $pdo;
11
12     public function __construct()
13     {
14         $this->pdo = Database::getInstance();
15     }
16
17     public function all(): array
18     {
19         $stmt = $this->pdo->query("SELECT * FROM prodi ORDER BY nama ASC");
20         return $stmt->fetchAll();
21     }
22
23     public function find(int $id): ?array
24     {
25         $stmt = $this->pdo->prepare("SELECT * FROM prodi WHERE id = :id");
26         $stmt->execute(['id' => $id]);
27         $row = $stmt->fetch();
28         return $row ? : null;
29     }
30
31     public function create(array $data): void
32     {
33         $stmt = $this->pdo->prepare("INSERT INTO prodi (kode, nama) VALUES (:kode, :nama)");
34         $stmt->execute($data);
35     }
36
37     public function update(int $id, array $data): void
38     {
39         $data['id'] = $id;
40         $stmt = $this->pdo->prepare("UPDATE prodi SET kode = :kode, nama = :nama WHERE id = :id");
41         $stmt->execute($data);
42     }
43
44     public function delete(int $id): void
45     {
46         $stmt = $this->pdo->prepare("DELETE FROM prodi WHERE id = :id");
47         $stmt->execute(['id' => $id]);
48     }
49 }

```

```

ACARAB > app > Models > MatakuliahModel.php
1  <?php
2
3  namespace App\Models;
4
5  use App\Core\Database;
6  use PDO;
7
8  class MatakuliahModel
9  {
10     private PDO $pdo;
11
12     public function __construct()
13     {
14         $this->pdo = Database::getInstance();
15     }
16
17     public function all(): array
18     {
19         $sql = "SELECT mk.*, p.nama AS prodi_nama
20                 FROM matakuliah mk
21                 JOIN prodi p ON mk.prodi_id = p.id
22                 ORDER BY mk.nama ASC";
23         $stmt = $this->pdo->query($sql);
24         return $stmt->fetchAll();
25     }
26
27     public function find(int $id): ?array
28     {
29         $stmt = $this->pdo->prepare("SELECT * FROM matakuliah WHERE id = :id");
30         $stmt->execute(['id' => $id]);
31         $row = $stmt->fetch();
32         return $row ? : null;
33     }
34
35     public function create(array $data): void
36     {
37         $stmt = $this->pdo->prepare(
38             "INSERT INTO matakuliah (kode, nama, sks, prodi_id) VALUES (:kode, :nama, :sks, :prodi_id)"
39         );
40         $stmt->execute($data);
41     }
42
43     public function update(int $id, array $data): void
44     {
45         $data['id'] = $id;
46         $stmt = $this->pdo->prepare(
47             "UPDATE matakuliah SET kode = :kode, nama = :nama, sks = :sks, prodi_id = :prodi_id WHERE id = :id"
48         );
49         $stmt->execute($data);
50     }
51
52     public function delete(int $id): void
53     {
54         $stmt = $this->pdo->prepare("DELETE FROM matakuliah WHERE id = :id");
55         $stmt->execute(['id' => $id]);
56     }
57 }

```

Setiap Model (MahasiswaModel, ProdiModel, MatakuliahModel) memanggil `Database::getInstance()` di constructor untuk mendapatkan koneksi PDO, kemudian menyediakan method `all()`, `find()`, `create()`, `update()`, dan `delete()`. Seluruh query yang melibatkan input dari pengguna (misalnya `id` pada `find()`, atau data form pada `create()`/`update()`) menggunakan `prepare()` dan `execute()` dengan *named parameter*, bukan menggabungkan string secara langsung.

3. CRUD Mahasiswa

SI Akademik

MahasiswaProdiMata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

NIM	Nama	Email	Prodi	Angkatan	Aksi
2401002	Ani Wijaya	ani@email.com	Teknik Informatika	2024	Edit Hapus
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	Edit Hapus
2402001	Citra Lestari	citra@email.com	Sistem Informasi	2024	Edit Hapus

SI Akademik

MahasiswaProdiMata Kuliah

Tambah Mahasiswa

NIM

Nama

Email

Prodi

-- Pilih Prodi --

Angkatan

2026

SI Akademik

MahasiswaProdiMata Kuliah

Edit Mahasiswa

NIM

2401002

Nama

Ani Wijaya

Email

ani@email.com

Prodi

Teknik Informatika

Angkatan

2024

`MahasiswaController` melengkapi seluruh alur CRUD: `index()` menampilkan daftar mahasiswa hasil `JOIN` dengan tabel `prodi`, `create()` menampilkan form tambah dengan dropdown `prodi`, `store()` memvalidasi input sebelum menyimpan, `edit()`

menampilkan form terisi data lama, `update()` memproses perubahan, dan `destroy()` menghapus data.

4. CRUD Prodi dan Mata Kuliah

SI Akademik		Mahasiswa	Prodi	Mata Kuliah
Data Prodi				+ Tambah Prodi
Kode	Nama	Aksi		
SI	Sistem Informasi	Edit	Hapus	
TI	Teknik Informatika	Edit	Hapus	
TK	Teknik Komputer	Edit	Hapus	

SI Akademik		Mahasiswa	Prodi	Mata Kuliah
Data Mata Kuliah				+ Tambah Mata Kuliah
Kode	Nama	SKS	Prodi	Aksi
TI102	Basis Data	3	Teknik Informatika	Edit Hapus
TI101	Pemrograman Dasar	3	Teknik Informatika	Edit Hapus
SI101	Pengantar SI	2	Sistem Informasi	Edit Hapus

Pola yang sama diterapkan pada `ProdiController` dan `MatakuliahController`. Pada `MatakuliahController`, daftar mata kuliah juga menampilkan nama prodi melalui `JOIN`, dan form tambah/edit menyediakan dropdown prodi seperti pada form mahasiswa.

5. Konfirmasi Hapus dengan JavaScript

Setiap tombol Hapus dibungkus dalam `<form method="POST">` dengan atribut `onsubmit="return confirm(...)"`. Apabila pengguna menekan "Batal" pada dialog konfirmasi, `confirm()` mengembalikan `false` sehingga `onsubmit` menghentikan pengiriman form, dan data tidak jadi terhapus.

6. Pengujian Relasi (Foreign Key Restrict)

SI Akademik		Mahasiswa	Prodi	Mata Kuliah
Data Prodi				+ Tambah Prodi
Kode	Nama	Aksi		
SI	Sistem Informasi	Edit	Hapus	
TI	Teknik Informatika	Edit	Hapus	

Dilakukan pengujian dengan mencoba menghapus data prodi yang masih memiliki mahasiswa terkait. Percobaan ini gagal karena constraint `ON DELETE RESTRICT` pada foreign key, dan ditangani pada `ProdiController::destroy()` dengan `try-catch` terhadap `PDOException`, menampilkan flash message "Prodi tidak bisa dihapus karena masih dipakai data lain." alih-alih menampilkan *fatal error* mentah.

7. Tugas Mandiri Fitur Pencarian

Ditambahkan form pencarian pada halaman index mahasiswa yang mengirim kata kunci melalui GET (`?q=...`). Pada `MahasiswaModel::all()`, apabila kata kunci tidak kosong, query ditambahkan klausa `WHERE nama LIKE :keyword OR nim LIKE :keyword`, dengan nilai `:keyword` diikat melalui prepared statement (`'%' . $keyword . '%'`), sehingga pencarian tetap aman dari SQL Injection meskipun menggunakan `LIKE`.

SI Akademik

MahasiswaProdiMata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

CariReset

NIM	Nama	Email	Prodi	Angkatan	Aksi
2401001	Budi Santoso	budi@email.com	Teknik Informatika	2024	Edit Hapus

SI Akademik

MahasiswaProdiMata Kuliah

Data Mahasiswa

+ Tambah Mahasiswa

CariReset

NIM	Nama	Email	Prodi	Angkatan	Aksi
Tidak ada data ditemukan.					

Pengujian dilakukan dengan memasukkan sebagian nama/NIM (hasil pencarian sesuai) dan mencoba string berbahaya seperti `' OR '1'='1` pada kolom pencarian (hasil tetap kosong/normal, membuktikan input tersebut diperlakukan sebagai data pencarian biasa, bukan perintah SQL).

E. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa pola Singleton pada class `Database` berhasil menggunakan satu koneksi PDO secara bersama sehingga penggunaan sumber daya lebih efisien. Penerapan prepared statement pada operasi CRUD dan pencarian `LIKE` juga membantu mencegah SQL Injection karena data pengguna dipisahkan dari struktur query. CRUD berhasil diterapkan pada tiga resource, yaitu Mahasiswa, Prodi, dan Mata Kuliah dengan struktur method yang konsisten dalam pola MVC.

Relasi One-to-Many antara Prodi dengan Mahasiswa dan Mata Kuliah berhasil ditampilkan menggunakan `JOIN`, sehingga nama prodi dapat ditampilkan secara informatif. *Foreign key* dengan `ON DELETE RESTRICT` juga berhasil menjaga integritas data dengan mencegah penghapusan prodi yang masih digunakan. Selain itu, konfirmasi JavaScript membantu mencegah penghapusan data secara tidak sengaja, sedangkan fitur pencarian menggunakan `LIKE` dan prepared statement memungkinkan pencarian data yang fleksibel sekaligus tetap aman dari SQL Injection.

